CVA6 on OpenROAD-flow-scripts: synthesis log

Oct 20, 2026 · @Someone

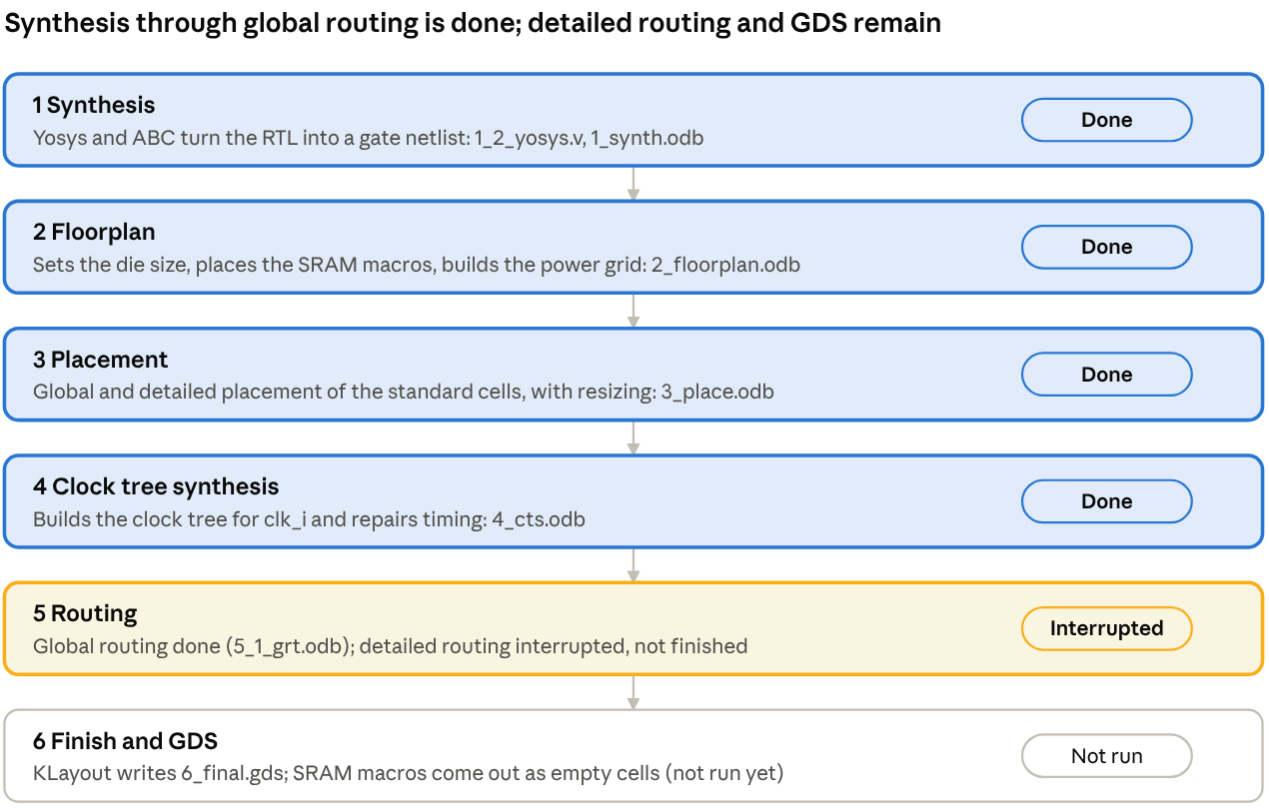
Summary

CVA6 now synthesizes on asap7 with hierarchical Yosys synthesis: 115,493 instances (8 of them SRAM macros), 18,297 μm², in 1 h 58 min. The original `make ... synth` failure came from the wrong Verilog frontend, not from the RTL.

The work moved through three attempts: a sky130hd port (stopped), a flat asap7 run (stopped), and the hierarchical asap7 run (finished). Only the hierarchical run completed, so there is no like-for-like timing comparison with the flat runs. Floorplan, placement, clock tree synthesis and global routing have since run on the hierarchical netlist. Detailed routing was interrupted by memory pressure and has not finished, so there is no GDS yet.

How the flow works

ORFS runs the design through six stages. Each is a Make target that reads the previous stage's database and writes the next, so a failure stops the chain, and Make reuses stages that are already built. The first five stages, through global routing, have run.

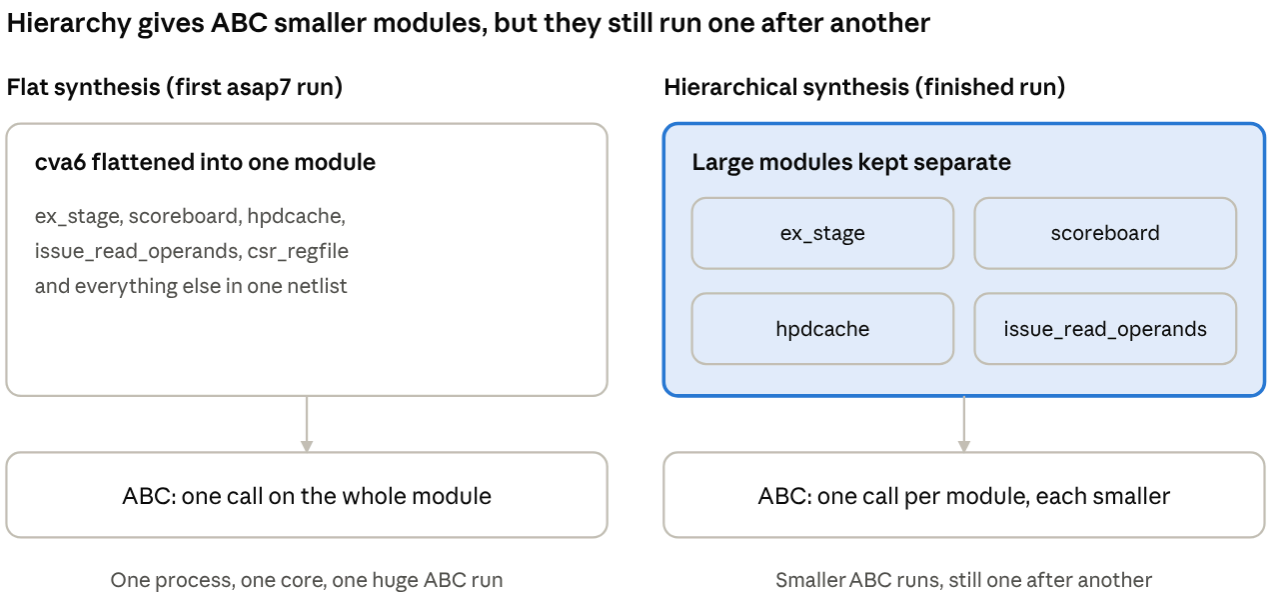


ORFS flow from RTL to GDS · 6 stages, status of each

Detailed routing was cut short by memory pressure on the machine, not by the design (see Detailed routing interruptions).

Flat vs hierarchical synthesis

The synthesis stage is the long one. In the flat run, Yosys merges the whole CPU into one module before ABC maps it. With `SYNTH_HIERARCHICAL = 1`, modules above the size threshold stay separate and ABC maps each one on its own.



flat vs hierarchical synthesis · how ABC is called

Both runs are one Yosys process, so the machine's other cores stay idle either way.

The original error

Yosys's built-in Verilog parser, selected by `READ_VERILOG_PARAMS="-sv"`, cannot read CVA6. The log stopped at `acc_dispatcher.sv:17` with `syntax error, unexpected TOK_IMPORT`, because CVA6 puts package imports in module headers.

Two more problems sat behind it:

- The config pointed at `designs/src/cva6_repo`, whose `cvfpu` and `hpdcache` submodules are empty.
- The sources instantiate `fakeram7_*` SRAM macros that exist only for asap7.

The fix is the `slang` SystemVerilog frontend (`SYNTH_HDL_FRONTEND = slang`), built into this Yosys (0.69+274), together with the ORFS-maintained sources in `designs/src/cva6`.

sky130hd attempt (stopped)

The sky130hd port got past the parser error but was never finished.

- **Config rewrite:** `designs/sky130hd/cva6/config.mk` now uses slang and the `designs/src/cva6` file list copied from asap7. The original is saved as `config.mk.orig`.
- **Stand-in RAMs:** `fakeram_behav.sv` replaces the four asap7 `fakeram7_*` macros with flop-based models, since sky130hd has no matching macros.

- **Probe ports:** `canonicalize.tcl` was copied from asap7 to delete the roughly 4k `rvfi_probes_o` debug ports.
- **First full run:** elaboration passed, then synthesis failed after 37 min with `Synthesized memory size 4096 exceeds SYNTH_MEMORY_MAX_BITS` (largest inferred memory 8192 bits).
- **Fix:** `SYNTH_MEMORY_MAX_BITS = 32768` was added to the config.
- **Rerun:** a clean rerun reached ABC mapping, but I stopped it after about 2 h, still unfinished.

With flop-based caches the netlist is very large and ABC runs on one flattened module, which is what made the run slow. The sky130hd files are still in place.

asap7 flat run (stopped)

ORFS already ships a working CVA6 config for asap7 (`designs/asap7/cva6/config.mk`), with `fakeram7_*` LEF and liberty views, so I used it unchanged. The macros stay blackboxed, so the caches do not become flops.

Elaboration plus canonicalize passed in about 95 s. The full flat synthesis was stopped at about 45 min, at pass 13 of the main synthesis and still before ABC mapping.

The stock config never set `SYNTH_HIERARCHICAL`, so this run was fully flat. Yosys's `abc` pass maps one module at a time, so a flattened CPU puts ABC on a single module and a single core (about 100% CPU on an 8-core machine). That is why I switched to hierarchical synthesis.

asap7 hierarchical run (finished)

The hierarchical run finished with exit code 0 and wrote `results/asap7/cva6/hier/1_2_yosys.v`.

`designs/asap7/cva6/config_hier.mk` includes the stock config and adds `FLOW_VARIANT = hier` (separate results directory), `SYNTH_HIERARCHICAL = 1`, and `SYNTH_MINIMUM_KEEP_SIZE = 10000` (stock value 40000), so modules above 10000 NAND2-equivalents stay separate.

Validation: Make resolves the variables correctly, and the log shows the "Keep modules above" message and the `KEEP_HIERARCHY` pass. The final stats list 93 modules.

Metric	Value
Instances in the OpenROAD database (8 are SRAM macros)	115,493
Cells in the top module alone (Yosys stat)	9,911
Area (μm²)	18,297
Main synthesis time (`1_2_yosys`)	7,079 s (1 h 58 min)
Canonicalize time	about 95 s
Peak memory	about 1 GB

The largest kept modules, by cells including submodules:

Module	Cells
`ex_stage`	28,146
`scoreboard`	27,005
`hpdcache`	26,877
`issue_read_operands`	18,429
`hpdcache_wbuf`	17,620
`hwpf_stride_wrapper`	11,799
`csr_regfile`	8,069
`instr_queue`	7,694

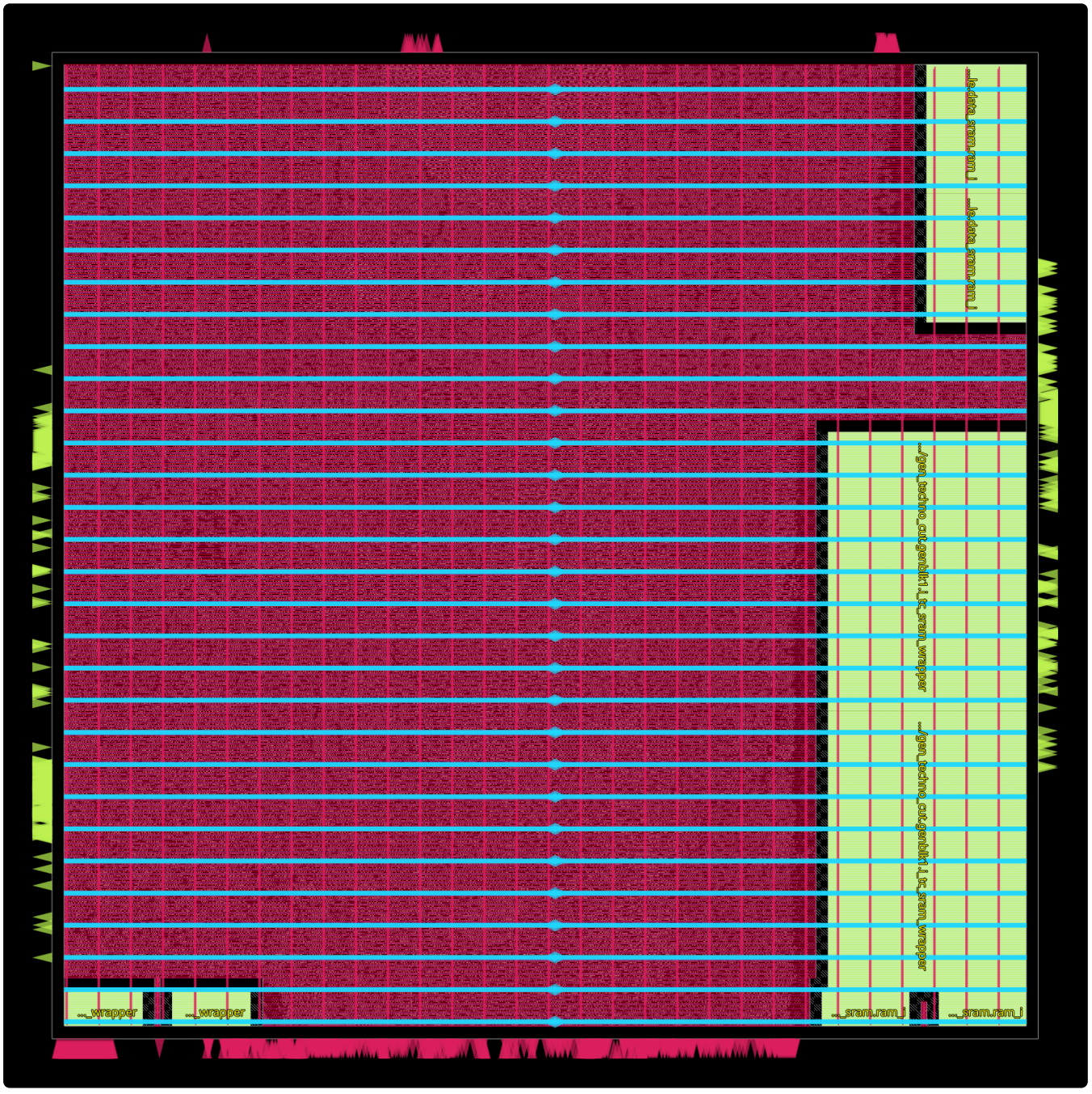
The count and area come from the synthesized OpenROAD database (`1_synth.odb`), where the 8 SRAM macros are one instance each and cover 4,167 μm². The Yosys hierarchy stat reads higher (141,810 cells, 21,219 μm²) because it also counts the alternative adder-architecture modules that remain in the netlist unused, so it overstates the design. Yosys still maps the kept modules one after another in a single process, so only one core is busy.

Floorplan and placement results

Floorplan and placement finished on the hierarchical netlist with the relaxed clock. Timing is met with a wide margin and there are no design-rule violations.

Stage	Instances	Area (µm²)	Utilization	Setup slack (ps)	Hold slack (ps)	Power (mW)
Synthesis	115,493	18,297	—	—	—	—
Floorplan	126,873	18,795	72.0%	759.4	5.9	35.5
Global placement	141,223	19,904	76.3%	3,340.9	16.6	29.1
Detailed placement	141,224	19,904	76.3%	3,338.4	16.4	29.2

After detailed placement the die is 27,448 µm² with a 26,095 µm² core, 964 I/O pins and an estimated wirelength of 556,059 µm. Setup, hold, max slew, max capacitance and max fanout violations are all zero.



The placed layout (`3_place.odb`): standard cells fill most of the die, the eight SRAM macros are the green blocks, and the horizontal cyan lines are the power grid.

Clock period. The constraint file `constraint_hier.sdc` sets `clk_period` to 5000, against 910 in the stock file. The first run at 910 started floorplan repair at a worst slack of -3,330 ps and made little progress, so I stopped it. Placement slack of +3,338 ps at 5000 puts the longest path at about 1.66 ns, so 5000 is looser than needed. My earlier estimate of a 4.2 ns critical path was wrong: it came from the first floorplan repair pass, before buffering and placement. A period near 2 ns looks reachable, but I have not tested it.

Clock tree synthesis and global routing results

Clock tree synthesis and global routing finished with timing still met and no setup or hold violations. Global routing reports 132 max-slew violations, which I have not examined.

Stage	Instances	Area (µm²)	Utilization	Setup slack (ps)	Hold slack (ps)	Power (mW)
Detailed placement	141,224	19,904	76.3%	3,338.4	16.4	29.2
Clock tree synthesis	142,416	20,423	78.3%	3,323.8	6.4	36.1
Global routing	142,416	20,423	78.3%	3,299.4	33.0	35.1

- **Clock tree:** TritonCTS found two clock nets on `clk_i`: one for the 8 SRAM macros and one for 9,434 register sinks. Clock skew is 50.6 ps for setup and 44.0 ps for hold after CTS. CTS added about 1,200 instances over placement.
- **Global routing:** 145,614 routed nets, 852,929 µm of wire and 1,201,807 vias. There are no antenna violations and no max-capacitance or max-fanout violations. It took 23 min 59 s, at about 394% CPU, with a 2.8 GB peak.

Detailed routing interruptions

Detailed routing, which produces `5_route.odb`, has not finished. It was interrupted four times, and the last two stops were caused by memory pressure on the machine, a VM with 10.8 GB of RAM. Routing itself used about 6.6 to 6.9 GB at the 10% mark of its first iteration, and each attempt that got going stopped at or shortly after that point.

Attempt	What happened	Cause
1	Stopped at 10% of iteration 0 when the earlier session ended.	I did not check the system log for this one.

Attempt	What happened	Cause
2	Failed after 1 second with exit code 127.	The flow looked for OpenROAD under `tools/install`, which does not exist, because `OPENROAD_EXE` was not set in the new session. The wrapper script now sets it.
3	Ran about 6 minutes, then died at 15:03.	`systemd-oomd` killed the Claude desktop app's process group under memory pressure, and the run was a child of that group.
4	Ran about 26 minutes, using 1 h 36 min of CPU on 4 threads, then was stopped at 15:35.	`systemd-oomd` acted at 15:34:52 and the desktop session was torn down, which stopped the run's own systemd unit. It had been marked `ManagedOOMPreference=avoid`, which did not protect it.

To get past this, the options are more RAM for the VM (16 GB or more), running the flow with the desktop session closed from a text console or SSH, or closing other heavy applications first. None of these has been tried.

Files and how to run

Paths are under `OpenROAD-flow-scripts/flow/` unless noted.

File	Change
`../../run_cva6_asap7.sh` (project root)	New wrapper script. Options: `-s STAGE`, `-H` (hierarchical), `-c` (clean), `-j N`. Logs to `run_logs/`.
`designs/asap7/cva6/config_hier.mk`	New hierarchical variant of the stock config.
`designs/sky130hd/cva6/config.mk`	Rewritten for slang, `designs/src/cva6` sources, and `SYNTH_MEMORY_MAX_BITS = 32768`.
`designs/sky130hd/cva6/config.mk.orig`	Backup of the original sky130hd config.
`designs/sky130hd/cva6/fakeram_behav.sv`	New flop-based stand-ins for the `fakeram7_*` macros.

File	Change
`designs/sky130hd/cva6/canonicalize.tcl`	Copied from asap7 to remove `rvfi_probes_o`.

To rerun the hierarchical synthesis from a clean state:

```
./run_cva6_asap7.sh -H -c
```

To continue past synthesis, add a stage, for example `-s floorplan`. Stages after synthesis are untested.

Open items and caveats

- **No speed comparison.** The flat asap7 run (about 45 min, still before ABC) and the flat sky130 run (about 2 h, unfinished) were both stopped, so I cannot say the hierarchical run is faster.
- **Single core.** Hierarchy gives ABC smaller modules but does not parallelize them. ORFS's parallel-synthesis route (`SYNTH_CHECKPOINT` with `SYNTH_BLACKBOXES`) is driven by Bazel and was not tried.
- **Latch warnings.** The log has many "Latch inferred" warnings from the Han-Carlson adder models in Yosys's own library files. I did not check whether they matter.
- **Detailed routing and GDS not done.** Floorplan, placement, clock tree synthesis and global routing have run, but detailed routing and the finish stage have not completed.
- **sky130hd left in place.** Its config edits and stand-in RAMs remain, and the sky130hd run never completed.
- **Relaxed clock.** The runs after synthesis use a 5000 ps clock period, not the stock 910 ps, and slack is about +3,300 ps. The timing numbers say little about what the design can reach.
- **Memory.** Detailed routing needs about 7 GB and was stopped twice by memory pressure on this 10.8 GB VM.

Next steps to GDS

Four stages are finished and detailed routing is the next one to complete. The steps below use the same wrapper with `-H`. Make reuses the finished stages, so a restart resumes from global routing (`5_1_grt.odb`).

1. **Free memory first.** Detailed routing needs about 7 GB, and the VM has 10.8 GB shared with the desktop. Add RAM, run without the desktop session, or close heavy applications, or it will be killed again.

2. **Detailed routing and fill:** `./run_cva6_asap7.sh -H -s route -j 4`. Check that the violation count reaches 0 and that `5_route.odb` is written.
3. **Finish and GDS:** `./run_cva6_asap7.sh -H -s final -j 4`. This writes the final reports and, through KLayout, `6_final.gds`.
4. **Review the results:** final setup and hold slack, power, and the 132 max-slew violations that global routing reported.
5. **Optional, tighten the clock:** try a period near 2 ns in `constraint_hier.sdc` and rerun from floorplan. I have not tested this.

Two points about the GDS step:

- **KLayout is installed.** It is version 0.28.16 at `/usr/bin/klayout`.
- **The SRAM macros have no GDS.** The platform has none for `fakeram7_*`, but `platforms/asap7/config.mk` sets `GDS_ALLOW_EMPTY ?= fakeram.*`, so the flow should accept them and write them as empty cells. I have not run the finish stage to confirm. For real shapes inside the macros, a KLayout script could convert each LEF to a placeholder GDS, or the FakeRAM generator may output GDS; I have not checked either.

To stop before GDS, `-s route` ends after detailed routing and fill, which gives a routed, timing-checked design as OpenROAD databases and does not need KLayout.